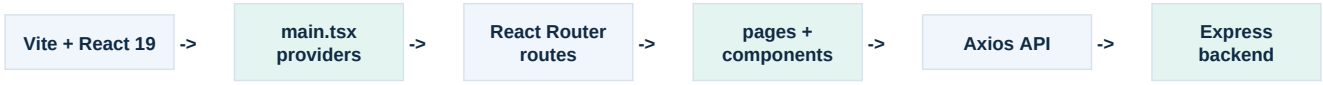
LaunchLens Frontend - Architecture, How It Works & React Interview Guide

A code-grounded guide to the current **frontend/** implementation. It explains the actual React application without code snippets.



The browser application is a Vite-built React/TypeScript single-page app. BrowserRouter chooses a page; pages compose reusable components; Axios sends cookie-enabled requests to the backend.

30-second explanation

LaunchLens has a public landing page, authentication screens, and an owner dashboard. React Router selects the screen, AuthContext restores the signed-in user from the server cookie, and ProtectedRoute blocks private pages until that check completes. Dashboard pages fetch projects, campaigns and analytics through one Axios client. Forms use React Hook Form plus Zod validation; charts render API analytics with Recharts; the AI panel requests structured campaign insights on demand.

Actual technology stack

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
Build/runtime	Vite 8, React 19, ReactDOM 19, TypeScript 6
Navigation	react-router-dom 7: BrowserRouter, Routes/Route, Navigate, NavLink, Link, useNavigate, useParams
Forms	react-hook-form 7 plus @hookform/resolvers and Zod 4
Data / auth	Axios 1 instance with localhost API base URL and withCredentials: true; Context + hooks
Visuals	Tailwind CSS 4 / Vite plugin, lucide-react icons, Recharts 3

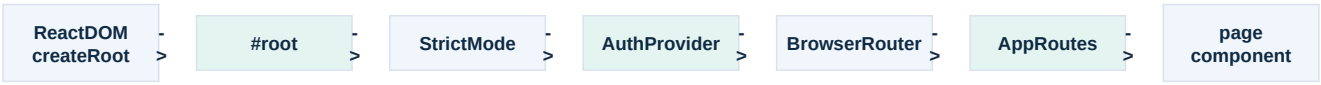
Actual route map

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
/	Home: Navbar, landing sections and Footer.
/login and /signup	AuthLayout wrapping LoginForm or SignupForm.
/dashboard	Protected Dashboard using useDashboard data hook.
/projects and /projects/:id	Protected project CRUD/list and selected project with campaign CRUD.
/campaigns/:id/analytics	Protected analytics dashboard with charts and AIInsights.

App composition, routing and protection

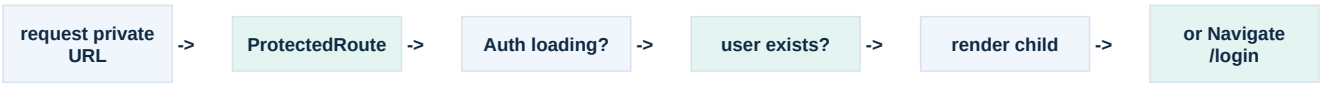
The frontend entry point mounts the entire application once into index.html #root. Provider order is intentional: AuthProvider surrounds BrowserRouter and App, so all routed pages can call useAuth.

Startup and route-selection flow



StrictMode is enabled in development to surface issues; it has no production behavior. App.tsx intentionally stays thin: it delegates route definition to routes/AppRoutes.tsx.

Protected-route flow



ProtectedRoute reads user and loading from AuthContext. While the initial GET /auth/me is pending it renders a loading screen, avoiding an incorrect early redirect. If no user is established it renders Navigate to /login with replace; otherwise it renders its children. This is UX routing protection, while the backend still enforces authorization for API calls.

Component boundaries that matter

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
main.tsx	Creates React root and composes StrictMode -> AuthProvider -> BrowserRouter -> App.
routes/AppRoutes.tsx	Declarative public and protected path-to-page map.
routes/ProtectedRoute.tsx	Waits for auth resolution, then renders child or redirects.
components/dashboard/DashboardShell.tsx	Reusable private-page layout with stateful mobile sidebar and Topbar slots.
components/dashboard/Sidebar.tsx	NavLink active styling, mobile overlay, profile menu, logout navigation and user initials.
components/dashboard/Topbar.tsx	Sticky title/subtitle/action header; opens the mobile navigation on small screens.
pages/Home.tsx	Composition page: Navbar + Hero + Problem + Solution + Workflow + CTA + Footer.

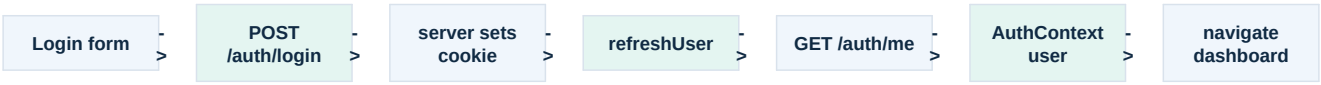
React concepts shown here

Composition: pages assemble focused presentational components. **Props:** DashboardShell receives title, subtitle, action and children; reusable chart/form components receive data and callbacks. **Conditional rendering:** loading, missing-data, modal, error and logged-in states determine what is shown. **Event handlers:** click and submit handlers update state or navigate. **Responsive UI:** Tailwind breakpoints switch sidebar behavior and grids rather than duplicating pages.

Authentication, shared state and API communication

The browser never manually stores a JWT in frontend state. The Axios client sends credentials; the server-set cookie is used by GET /auth/me and protected API calls.

Sign-in and session restoration



LoginForm performs login, then refreshUser fetches the authoritative user profile into Context, then navigation goes to /dashboard. SignupForm instead posts account data, shows a browser alert, and routes to /login; it does not establish session state itself.

AuthContext state machine

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
State	user starts as null; loading starts true. Context exposes user, loading, refreshUser and logout.
Initial effect	On provider mount, refreshUser calls GET /auth/me. Success stores response.data.data; failure keeps user null; finally clears loading.
Logout	POST /auth/logout clears server cookie, then the frontend sets local user to null and Sidebar navigates to /login.
Why Context	One provider state prevents each page/component independently asking who is logged in; Sidebar and ProtectedRoute share the same user source.
useAuth	A custom hook wraps useContext and throws if used outside AuthProvider, making incorrect provider placement fail clearly.

Axios contract

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
api/axios.ts	One Axios instance: baseURL http://localhost:5000/api, JSON header, withCredentials true.
Why withCredentials matters	It permits browser requests to carry the httpOnly authentication cookie to the local API; this matches backend CORS credentials configuration.
Current limitation	The frontend base URL is hard-coded to localhost:5000. It needs environment-based configuration for a deployed frontend.
Error treatment	Auth forms use alert messages; dashboard hook and AI panel have user-visible error state; several CRUD pages log failures only. Error presentation is therefore not yet uniform.

Data-fetching patterns

useEffect: kicks off data loading after a component mounts or its ID changes. **useState:** stores response data, loading flags, modal state, selection and errors. **useCallback:** stabilizes Projects.fetchProjects so its effect can depend on it. **Promise.all:** ProjectDetails loads the project and its campaign list in parallel before clearing the page loading state. The package includes TanStack React Query, but the inspected current source uses local hooks/state and Axios directly, not React Query.

Projects and campaigns: interactive UI data flow

Projects.tsx and ProjectDetails.tsx are stateful orchestration pages. They keep fetched lists and selected records locally, while modals receive those records and submit validated data upward through callbacks.

Project management flow

Projects shows Loader until the list request ends, then either EmptyState or ProjectGrid. Creating, updating and deleting each call the relevant endpoint, refetch the list after success, and close/reset the relevant dialog state. Opening a card uses useNavigate to /projects/:id.

Project detail and campaign flow

ProjectDetails reads the path parameter with useParams. It independently tracks page loading and campaign loading. The selected campaign plus mode controls one shared CampaignFormModal for create and edit; DeleteCampaignDialog separates destructive confirmation from the list.

Forms and validation

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
React Hook Form	LoginForm, SignupForm, ProjectFormModal and CampaignFormModal call useForm. register binds fields, handleSubmit gates submission, errors render field feedback, and isSubmitting disables/shows button loading.
Zod + zodResolver	Schemas validate browser input before the HTTP request and infer the form TypeScript type. Login uses email + password minimum 6; signup min 3 name / valid email / min 8 password; project and campaign schemas validate names and URLs.
Modal reset behavior	Project/Campaign forms useEffect when opened or when mode/record changes. In edit mode they reset defaults from the selected record; otherwise reset blanks. Close and successful submit reset the form.
Server remains authoritative	Frontend validation helps feedback and avoids avoidable requests; the backend repeats validation and authorization. Client validation alone is not a security boundary.

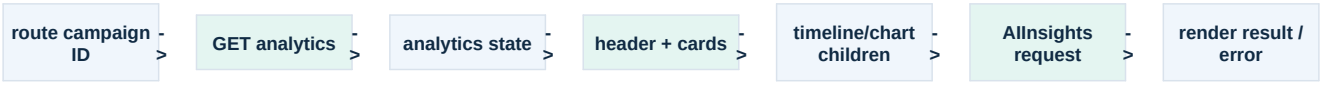
Useful React concepts to explain

Controlled workflow, not necessarily controlled inputs: React Hook Form manages field registration and submission while component state controls whether a modal is present. **Lifting state:** Projects owns selectedProject, mode and modal state because ProjectGrid/cards need to request actions and form/dialog components need the result. **Derived UI:** the same modal renders create or edit labels based on mode, rather than maintaining two duplicated forms.

Analytics and AI presentation layer

CampaignAnalytics.tsx transforms a campaign ID from the route into a dashboard of server-provided aggregates. Its child components specialize visualization or display, keeping the page focused on request/lifecycle orchestration.

Analytics page flow

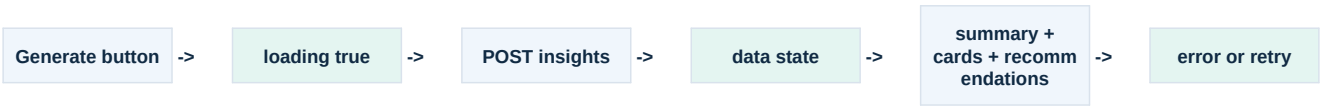


The initial effect refetches when id changes. While pending it shows Loader; absent analytics produces a fallback message. On success it passes the relevant slices to presentation components.

What each visualization actually does

FILE / CONCEPT	ACTUAL ROLE / INTERVIEW EXPLANATION
AnalyticsHeader / AnalyticsCards	Show campaign metadata and summary/device-oriented aggregate information received from the API.
TimelineChart	Formats dates through utils/dateUtils, then renders Recharts AreaChart within ResponsiveContainer; shows EmptyAnalyticsState when no data.
DeviceChart	Renders a Recharts PieChart with device name/count keys and a rotating fixed color palette.
AnalyticsBarChart	Reusable Recharts BarChart. CampaignAnalytics supplies it for Top Referrers and Browsers with distinct x keys and gradients.
ChartTooltip / AnalyticsSection	Shared tooltip and section wrappers keep chart presentation consistent.
ResponsiveContainer	Lets Recharts measure its containing element, so chart width responds to the page layout rather than using a fixed pixel width.

AI insights interaction



AIInsights does not generate automatically. The user starts the POST /campaigns/:id/insights request. It disables the button during loading, renders a spinner, stores successful structured data, and maps insight/recommendation arrays into cards. Priority maps to high/medium/low visual classes. A failure stores a visible retry message.

Current frontend scope and limits

Countries are not rendered in CampaignAnalytics or passed to AIInsights, even though the backend analytics type includes countries. AI results are held only in component state and disappear on a page reload; no caching or insight history is implemented in the inspected frontend. The analytics response is currently stored as **any** in CampaignAnalytics, so its response shape is not fully TypeScript-enforced at that boundary. There is no global toast/error system; several CRUD request failures are only logged to the console.

Interview-ready explanation and questions

These answers describe the implementation as it stands. They deliberately distinguish current behavior from installed-but-unused or not-found capabilities.

60-90 second technical explanation

LaunchLens is a Vite and React TypeScript SPA. main.tsx mounts the app inside StrictMode, AuthProvider and BrowserRouter. Public routes render the landing and auth screens; private routes go through ProtectedRoute, which waits for AuthContext to restore the cookie-backed server session. A shared Axios client sends credentialed JSON calls to the Express API. The dashboard uses local React state and effects to fetch data, manage loading/errors and drive project/campaign modals. React Hook Form and Zod validate input before requests. The analytics page sends a campaign ID to the API and renders returned aggregates through reusable Recharts components; its AI panel requests and maps structured insights only when the user asks.

Two-minute user journey

A visitor can view the composed landing page at /. A returning owner opens a private URL: AuthProvider performs GET /auth/me and ProtectedRoute shows Loading until it knows whether user exists. Login sends credentials, then refreshes the user profile and navigates to dashboard. Dashboard uses useDashboard to fetch stats. In Projects, the page owns project list and selection state; a modal validates project input, posts or patches the API, then refetches so the screen matches server truth. Opening a project loads project details and campaigns concurrently. The campaign analytics page fetches aggregates, passes each slice to its chart/card component, and optionally invokes the AI insight endpoint. Logout calls the API, clears Context user, and returns to login.

15 realistic frontend interview questions

Why React Context for auth?	It centralizes user/loading/session actions for all routes and layout components.	**Deeper:** It avoids duplicate current-user requests in every component.
How are private pages protected?	ProtectedRoute waits for auth loading, renders children for a user, otherwise Navigate to login.	**Deeper:** Backend authorization still protects every API resource.
Why withCredentials?	The API client must carry the server-set httpOnly cookie.	**Deeper:** It matches the backend CORS credentials configuration.
What does useEffect do here?	It starts fetches after mount or ID changes, and restores the session once.	**Deeper:** It is also used to reset modal forms for create/edit state.
Why useCallback in Projects?	It stabilizes fetchProjects for useEffect dependencies.	**Deeper:** It also lets CRUD handlers reuse one refresh routine.
Why React Hook Form?	It manages registration, errors, submission and isSubmitting for forms.	**Deeper:** Zod resolver makes validation and inferred types share one schema.
Why validate on frontend and backend?	Frontend improves feedback; backend is the security/authority boundary.	**Deeper:** Browser checks can be bypassed, so both are needed.
How does a project update appear?	The page patches, refetches projects, then closes/resets modal state.	**Deeper:** Refetch favors server truth over manually merging state.
Why one modal for create/edit?	Mode and selected record change labels/defaults while sharing UI/validation.	**Deeper:** It reduces duplicate form logic.
How are charts responsive?	Each Recharts chart sits in ResponsiveContainer.	**Deeper:** The container adapts as the dashboard grid changes.
What happens on AI request?	AIInsights toggles loading, posts by campaign ID, then maps structured response data.	**Deeper:** Failure shows a retry message; it does not auto-run.
What state is global?	Only auth user/loading/actions are Context global.	**Deeper:** Lists, modals and analytics remain local to their pages.
Is React Query used?	It is installed but not used in inspected source.	**Deeper:** Current requests use Axios plus useState/useEffect directly.
What are frontend limits?	Hard-coded localhost API base URL, inconsistent error UX, no AI cache/history.	**Deeper:** Analytics response uses any at one page boundary.
How is mobile navigation handled?	DashboardShell stores mobileMenuOpen and passes callbacks to Sidebar/Topbar.	**Deeper:** Sidebar uses a fixed overlay/drawer on small screens, static on md+.